DSA Coursework Report
Assignment 1: Arrays and Assignment 2: Linked Lists

Student Topic Context

This report documents the completed Python implementations for university course registration analytics and a helpdesk ticket queue. The solution files are:

- Assignment 1- Arrays (1).py
- Assignment 2- Linked Lists (1).py

Assignment 1: University Course Registration Analytics

The program stores registrations in a Python list where each integer is a student ID. It implements four required operations and prints the expected sample output:

Most frequent student: 1023
Unique registrations: [1023, 1050, 1102, 1201, 1300]
First non-repeated student: 1201
Subarray with target sum exists: True

The implementation also includes assertion-based tests for normal inputs, empty lists, single-element lists, repeated-only lists, and a list containing negative values for the subarray sum algorithm.

Assignment 1 Algorithm Explanation

1. Most Frequent Registrant
A dictionary stores how many times each student ID appears. While scanning the list, the algorithm updates the current most frequent ID whenever a higher count is found. If the input list is empty, it returns None.
Worst-case time complexity: O(n)
Worst-case space complexity: O(n)

2. Ordered Deduplication
A set tracks IDs that have already appeared, and a result list stores only the first occurrence of each ID. This preserves the original first-appearance order.
Worst-case time complexity: O(n)
Worst-case space complexity: O(n)

3. First Unique Student
The list is scanned once to count all student IDs, then scanned again to return the first ID whose count is exactly one. If no such ID exists, it returns None.
Worst-case time complexity: O(n)
Worst-case space complexity: O(n)

4. Contiguous Subarray Sum
The algorithm uses prefix sums. If current_sum - target has appeared before, then the values between the previous prefix and the current position sum to the target.
Worst-case time complexity: O(n)
Worst-case space complexity: O(n)

Assignment 2: University Helpdesk Ticket Queue

The linked list program defines a Ticket node containing ticket_id, student_name, issue, and next. The TicketQueue class stores head and tail references.

Implemented Operations

1. Enqueue Ticket: adds a ticket to the end of the queue. A tail pointer makes this O(1).
2. Priority Insert: searches for a specified ticket ID and inserts a new ticket immediately after it.
3. Resolve Ticket: removes a ticket with the given ticket ID, including correct handling for head, tail, empty queue, and missing ticket cases.
4. Find Middle Ticket: uses slow and fast pointers to find the middle ticket in one traversal.
5. Reverse Queue: reverses the queue in place by changing next pointers without copying node data.
6. Display Queue: traverses and prints each ticket's ID, student name, and issue.

The file includes assertion-based tests for empty queues, single-ticket queues, priority insertion, deletion, middle finding, and reversal.

Assignment 2 Complexity Analysis

Enqueue Ticket
Worst-case time complexity: O(1) with the maintained tail pointer
Worst-case space complexity: O(1)

Priority Insert
Worst-case time complexity: O(n), because the specified ticket may be at the end or absent
Worst-case space complexity: O(1)

Resolve Ticket
Worst-case time complexity: O(n), because the ticket may be at the end or absent
Worst-case space complexity: O(1)

Find Middle Ticket
Worst-case time complexity: O(n), completed in one traversal pass using fast and slow pointers
Worst-case space complexity: O(1)

Reverse Queue
Worst-case time complexity: O(n)
Worst-case space complexity: O(1)

Display Queue
Worst-case time complexity: O(n)
Worst-case space complexity: O(1)

Bonus Challenge: Python List vs Singly Linked List

If the helpdesk performs thousands of searches per day but only a few insertions and deletions, a Python list is generally more appropriate than a singly linked list.

Both a Python list and a singly linked list have O(n) worst-case linear search when searching by ticket ID without an index. However, Python lists store references in a contiguous array, so scanning them is usually faster in practice because nearby elements are loaded efficiently by the CPU cache. A linked list requires pointer chasing from node to node, and each node may be stored in a different memory location. That pattern causes more cache misses and slower real-world traversal.

Python lists also avoid per-node pointer overhead. A linked list node stores ticket data plus an extra next pointer, and each node is separately allocated. A Python list uses one contiguous block of references to ticket objects, which tends to reduce allocation overhead and improve memory access behavior. Therefore, for search-heavy workloads with few structural changes, the Python list is usually the better practical choice.